Machine Learning
(6CS4-02)

Unit-4 Notes

Vision of the Institute

To become a renowned centre of outcome based learning and work towards academic, professional, cultural and social enrichment of the lives of individuals and communities.

Mission of the Institute

M1- Focus on evaluation of learning outcomes and motivate students to inculcate research aptitude by project based learning.
M2- Identify, based on informed perception of Indian, regional and global needs, the areas of focus and provide platform to gain knowledge and solutions.
M3- Offer opportunities for interaction between academia and industry.
M4- Develop human potential to its fullest extent so that intellectually capable and imaginatively gifted leaders can emerge in a range of professions.

Vision of the Department

To become renowned Centre of excellence in computer science and engineering and make competent engineers & professionals with high ethical values prepared for lifelong learning.

Mission of the Department

M1-To impart outcome based education for emerging technologies in the field of computer science and engineering.
M2-To provide opportunities for interaction between academia and industry.
M3- To provide platform for lifelong learning by accepting the change in technologies
M4- To develop aptitude of fulfilling social responsibilities.

Program Outcomes (PO)

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis**: Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Educational Objectives (PEO)

1. To provide students with the fundamentals of Engineering Sciences with more emphasis in Computer Science &Engineering by way of analyzing and exploiting engineering challenges.
2. To train students with good scientific and engineering knowledge so as to comprehend, analyze, design, and create novel products and solutions for the real life problems.
3. To inculcate professional and ethical attitude, effective communication skills, teamwork skills, multidisciplinary approach, entrepreneurial thinking and an ability to relate engineering issues with social issues.
4. To provide students with an academic environment aware of excellence, leadership, written ethical codes and guidelines, and the self-motivated life-long learning needed for a successful professional career.
5. To prepare students to excel in Industry and Higher education by Educating Students along with High moral values and Knowledge

Program Specific Outcomes (PSO)

PSO1: Ability to interpret and analyze network specific and cyber security issues, automation in real word environment.

PSO2: Ability to Design and Develop Mobile and Web-based applications under realistic constraints.

Course Outcome:

CO1: Understand the concept of machine learning and apply supervised learning techniques.
CO2: Illustrate various unsupervised leaning algorithm for clustering, and market basket analysis.
CO3: Analyze statistical learning theory for dimension reduction and model evaluation in machine learning.
CO4: Apply the concept of semi supervised learning, reinforcement learning and recommendation system.

CO-PO Mapping:

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
Understand the concept of machine learning and apply supervised learning techniques.	3	3	3	3	2	1	1	1	1	2	1	3
Illustrate various unsupervised leaning algorithm for clustering, and market basket analysis.	3	3	3	2	2	1	1	1	1	1	1	3
Analyze statistical learning theory for dimension reduction and model evaluation in machine learning.	3	3	3	3	2	2	2	2	1	2	2	3
Apply the concept of semi supervised learning, reinforcement learning and recommendation system.	3	3	3	3	2	1	1	1	1	2	1	3

SYLLABUS:

RAJASTHAN TECHNICAL UNIVERSITY, KOTA

Syllabus

III Year-VI Semester: B.Tech. Computer Science and Engineering

6CS4-02:Machine Learning

Credit: 3 **Max. Marks: 150(IA:30, ETE:120)**

3L+0T+0P **End Term Exam: 3 Hours**

SN	Contents	Hours
1	**Introduction:** Objective, scope and outcome of the course.	01
2	**Supervised learning algorithm:** Introduction, types of learning, application, Supervised learning: Linear Regression Model, Naive Bayes classifier Decision Tree, K nearest neighbor, Logistic Regression, Support Vector Machine, Random forest algorithm	09
3	**Unsupervised learning algorithm:** Grouping unlabelled items using k-means clustering, Hierarchical Clustering, Probabilistic clustering, Association rule mining, Apriori Algorithm, f-p growth algorithm, Gaussian mixture model.	08
4	**Introduction to Statistical Learning Theory**, Feature extraction - Principal component analysis, Singular value decomposition. Feature selection – feature ranking and subset selection, filter, wrapper and embedded methods, Evaluating Machine Learning algorithms and Model Selection.	08
5	**Semi supervised learning, Reinforcement learning:** Markov decision process (MDP), Bellman equations, policy evaluation using Monte Carlo, Policy iteration and Value iteration, Q-Learning, State-Action-Reward-State-Action (SARSA), Model-based Reinforcement Learning.	08
6	**Recommended system,** Collaborative filtering, Content-based filtering Artificial neural network, Perceptron, Multilayer network, Backpropagation, Introduction to Deep learning.	08
	Total	42

LECTURE PLAN:

Unit No./ Total Lecture Reqd.	Topics	Lect. Reqd.	Lect. No.
Unit-I (10)	1. Introduction to subject and scope	1	1
	2. Introduction to learning, Types of learning and Applications	1	2
	3. Supervised Learning	1	3
	4. Linear Regression Model	1	4
	5. Naïve Bayes Classifier	1	5
	6. Decision Tree	1	6
	7. K-nearest Neighbor	1	7
	8. Logistic Regression	1	8
	9. Support Vector Machine	1	9
	10. Random Forest Algorithm	1	10
BC-1	**Gradient Descent**	1	11
Unit-II (8)	1. Introduction to clustering, K-mean clustering	2	12
	2. Hierarchical Clustering	1	14
	3. Probabilistic Clustering	1	15
	4. Association Rule Mining	1	16
	5. Apriori Algorithm	1	17
	6. f-p Growth Algorithm	1	18
	7. Gaussian Mixture Model	1	19
Unit-III (8)	1. Feature Extraction- PCA and SVD	3	22
	2. Feature Selection- Feature Ranking and Subset Selection	2	24
	3. Filter, Wrapper and Embedded Methods	1	25
	4. Evaluating Machine Learning Algorithms	1	26
	5. Evaluating Model Selection	1	27
Unit- IV (8)	1. Semi supervised learning: Markov Decision Process (MDP)	2	29
	2. Bellman Equations	1	30
	3. Policy Evaluation using Monte Carlo	1	31
	4. Policy iteration and Value iteration	1	32
	5. Q-Learning	1	33
	6. State-Action-Reward-State-Action (SARSA)	1	34
	7. Model-based Reinforcement Learning	1	35

Unit- V (8)	1. Recommendation system: Collborative Filtering	1	36
	2. Content based filtering	1	37
	3. Artificial neural network	1	38
	4. Perceptron	1	39
	5. Multilayer network	1	40
	6. Backpropagation	1	41
	7. Introduction to Deep learning.	2	42
BC-2	**Genetic Algorithms**	1	44

Text Book:
Machine learning- Tom M Mitchell

✱ Semi- Supervised Learning :-

→ The basic difference between Supervised & Unsupervised Learning is that supervised learning datasets have an output label associated with each tuple while Unsupervised Learning datasets donot.

→ In any supervised learning algorithm the dataset has to be hand labeled either by machine learning Engineer or a data Scientist. & that is very costly process
& basic disadvantage of unsupervised learning is that It's application spectrum is limited.

→ So to counter these disadvantage, we use the concept of semi- supervised learning, In which algorithm is trained upon a combination of labeled & unlabeled data.

→ It contain a very small amount of labeled data & a very large amount of unlabeled data.

→ first, the programmer will cluster similar data using an unsupervised learning algorithm and then use the existing labeled data to label the rest of the unlabeled data.

Scanned with CamScanner

Machine Learning (Unit-4)

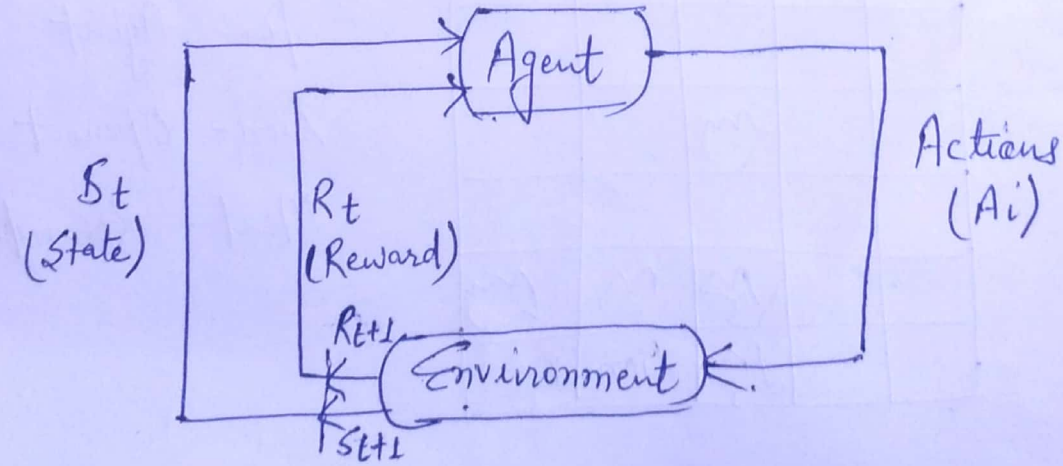
* Semi Supervised Learning, Reinforcement Learning :-

→ It is a type of machine learning technique that enables an agent to learn in an interactive environment by trial & error using feedback from its own actions and experiences.

→ Learn from mistakes.

→ Both supervised and reinforcement learning use mapping between i/p & o/p, unlike supervised learning where the feedback provided to the agent is correct set of actions for performing a task, reinforcement learning uses rewards and punishment as signals for positive & negetive behaviours.

→ As compared to unsupervised learning, reinforcement learning is different in terms of goals, while goal in unsupervised learning is to find a suitable action model that would maximize the total cumulat-ive reward of the agent.



Scanned with CamScanner

1) Environment - Physical world in which the agent operates. (4)

(6) Agent

2) State - Current situation of the agent.

3) Reward - Feedback from the environment.

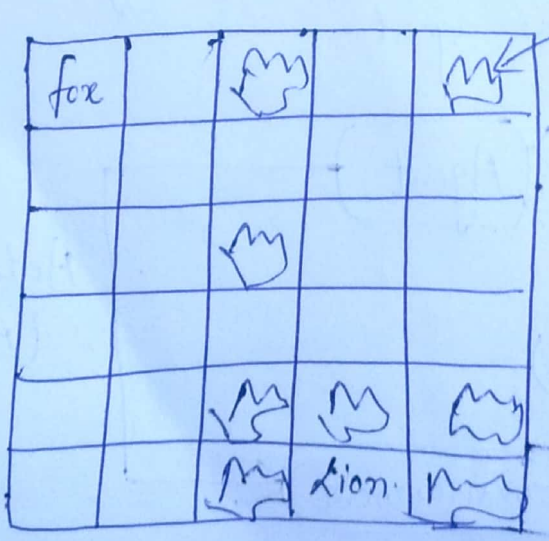
4) Policy - Method to map agent's state to actions.

5) Value - Future reward that an agent would receive by taking an action in particular state.

(+ Policy - The approach that the agent uses to determine the next action based on the current state)

* Reward Maximization :- This theory states that an RL agent must be trained in such a way that he takes the best action so that the reward is maximum.

Example



fox - Agent

Lion - Opponent

Meat - Reward

Scanned with CamScanner

Topic: Reward Maximization

→ End goal of fox is to eat maximum amount of meat before being eaten by the tiger.

→ Since a fox is clever, he eats the meats closer to rather the meat which is close to the lion.

→ Since ~~not~~ closer to the tiger higher the chances of getting killed.

→

★ Exploration and Exploitation :-

→ Exploitation is about using the already known exploited information to heighten the rewards.

→ Exploration is about exploring & capturing ~~home~~ more information about an environment

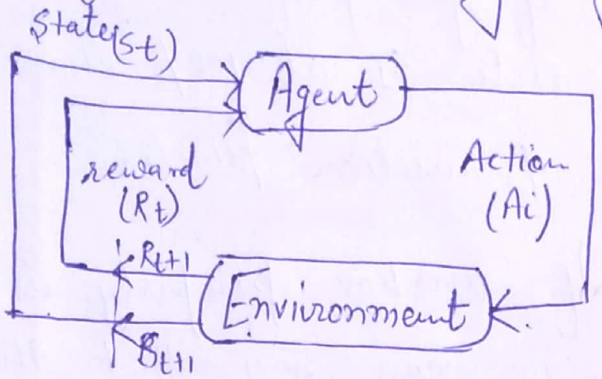
→ As in previous example, if fox eats the meat chunks closer to him, not that are closer to tiger, even though they might give him big rewards, this is what exploitation (use currently known Infor^n).

→ but if the fox decides to explore a bit It can find a bigger reward that is big chunks of meat, that is exactly what exploration is.

Scanned with CamScanner

Markov Decision Process :-

→ It is a mathematical framework behind any Reinforcement learning algorithm.



→ It is a tuple (S, A, P, R, γ)

→ MDP describes a problem, not a solution to that problem. 4 objects (to formallize RL Problem):

→ S: State Space (Set of states $S_0, S_1 \ldots$)

A: Action Space (Set of Actions).

T: $S \times A \rightarrow \pi(S)$: Transition function $\rightarrow P(s', s|a)$

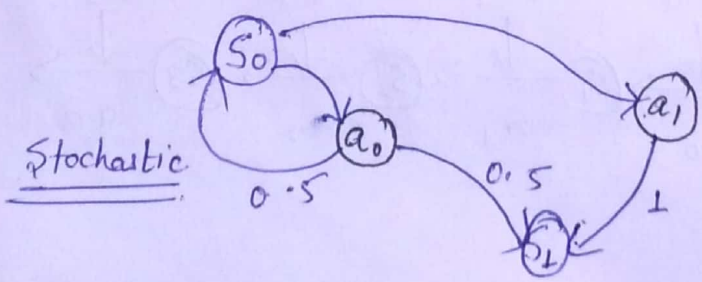
(# What is the probability of being in a next state given the current state & current action).

r: $S * A \rightarrow \mathbb{R}$: reward function

(# The value that you get performing one particular action in one particular state

If value is +ive → Reward
" " -ive → Punishment

☆ Stochastic VS Deterministic Transition function :-



Scanned with CamScanner

→ At state S_0 if you perform action a_0, 50% chances of go to state S_0 & 50% chances of go to state S_1 that is stochastic

→ but At state S_0 if you perform action a_1 you will reach to state S_1 at 100% chances is called deterministic transition function.

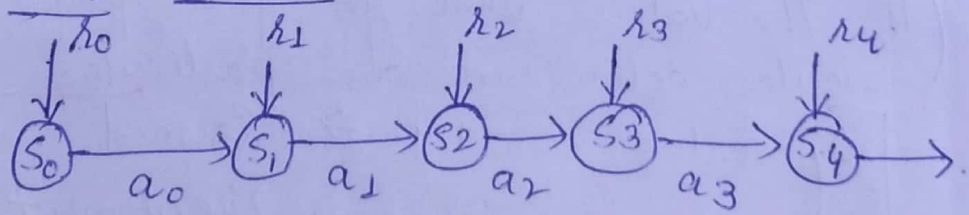
★ Markov Property :- If markov property is associated with states then we can say that the state transition based on the action on the state depends only on current state of the system, It doesn't depend on the entire state - action trajectory that happened in the past.

Example :- (chess board) (tic-tac-toes) games.

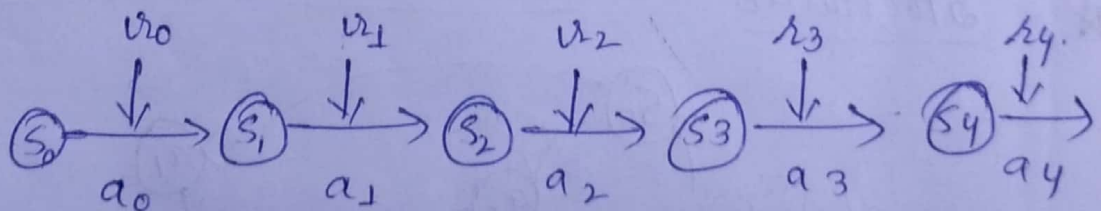
$$T(s^t, a^t, s^{t+1}) = p(s' | s, a).$$

★ Rewards :- over action or over states.

✦ Reward over states



✦ Reward over actions in states.



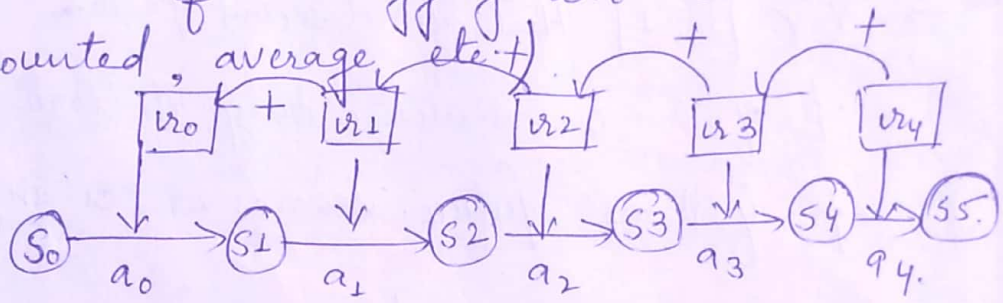
Scanned with CamScanner

Topic : Markov Decision Process

Goal :- is to find a policy $\pi : S \to A$ maximizing an aggregation of rewards on the long run.

✱ Aggregation Criterion : Mere Sum

→ The computation of value functions assumes the choice of an aggregation criterion (discounted, average, etc..)

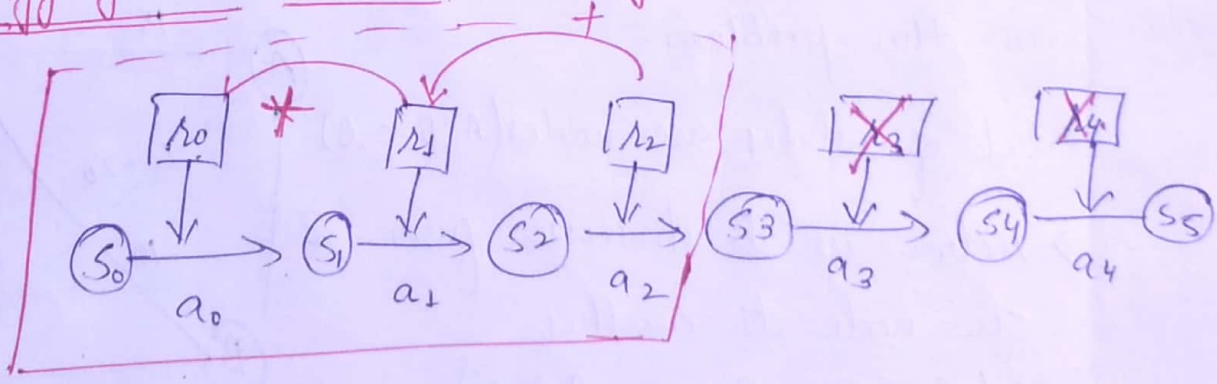


→ Mere sum (finite horizon N:)

$$V^{\pi}(s_0) = r_0 + r_1 + r_2 + r_3 + \dots r_N.$$

→ The sum over a infinite horizon may be infinite

✱ Aggregation Criterion : Average Over a Window

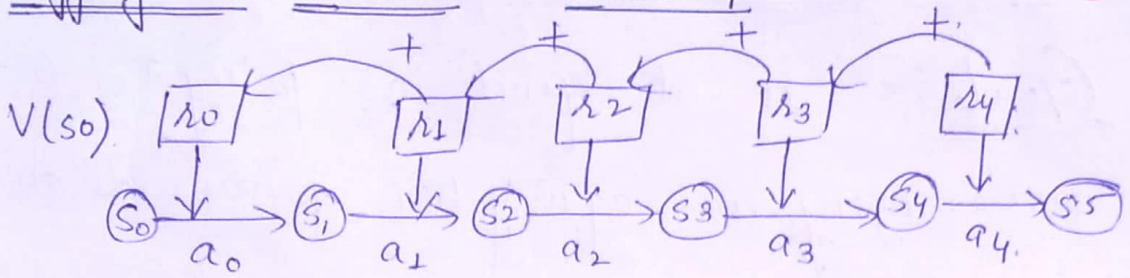


Average criterion on a window

$$V^{\pi}(s_0) = \frac{r_0 + r_1 + r_2}{3}$$

Scanned with CamScanner

✱ Aggregation criterion : Discounted



→ Discounted Criterion.

$$V^{\pi}(s_0) = \sum_{t=t_0}^{\infty} \gamma^t r(s_t, \pi(s_t))$$

→ $\gamma \in [0, 1]$ that is discount factor

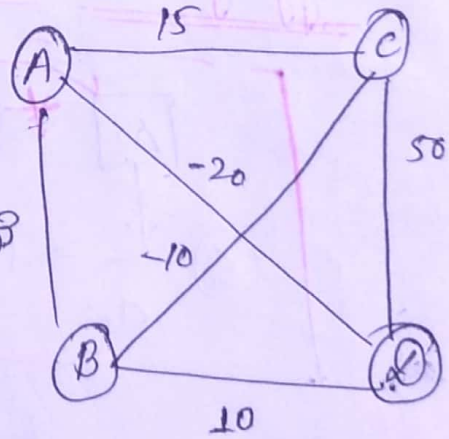
- if $\gamma = 0$ sensitive only to immediate reward.
- if $\gamma = 1$ future rewards are as imp as ↑

→ It is most used case.

Example :-

Goal: Find the shortest path between A & D with minimum possible cost

in this problem.



→ Set of states are nodes (A, B, C, D)

→ Action is to traverse from one node to another (A→B, B→C, A→C)

→ Reward is cost represented by each edge

→ Policy is path taken to reach the destination with maximum reward (A → C → D).

Scanned with CamScanner

★ BellMan Equations

→ The goal of reinforcement learning is :

Given the current state we are in, choose the optimal action which will maximize the long-term expected reward provided by the environment.

→ Dr. Richard Bellman is father of dynamic Programming & that is

A class of algorithms which seek to satisfy complex problems by breaking them into smaller subproblems & solving these subproblems recursively.

Why Bellman Eqn.

✦ Given the state I'm in, assuming I take the best possible action now & at each ~~step~~ subsequent step, what long-term reward can I expect ?

✦ OR what is the VALUE of the STATE ?

✦ It helps us to evaluate the expected reward relative to the advantage or disadvantage of each state.

(+ive Reward) (-ive Reward).

Terminology :-

Agent → Policy : $P(a/s)$

(in particular state what action to perform to get max. reward)

Scanned with CamScanner

Environment :- Transition dynamics

$$P(s', r \mid s, a)$$

s' ↑ next state; s ↑ current state

State Transition Probability

$$= \sum_{r \in R} P(s', r \mid s, a)$$

Total discounted return :-

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \gamma^{N-1} R_{t+N}.$$

where $\gamma \in [0, 1]$ → discount factor.

Value of state :-

(with respect to a policy π).

$$V_\pi(s) = E_\pi [G_t \mid S_t = s].$$

↑ (Total expected discounted Reward.)

$$\Rightarrow V_\pi(s) = E_\pi [G_t \mid S_t = s]$$

$$\Rightarrow \quad " \quad = E_\pi [R_{t+1} + \gamma G_{t+1} \mid S_t = s].$$

$\because G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$

$\Rightarrow G_t = R_{t+1} + \gamma [R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \ldots]$

$\Rightarrow G_t = R_{t+1} + \gamma (G_{t+1})$

Scanned with CamScanner

Topic : Bellman Equation ...

$$V_\pi(s) = E_\pi \left[R_{t+1} + \gamma G_{t+1} \mid S_t = s \right].$$

$$\Rightarrow V_\pi(s) = \sum_a \pi(a/s) \sum_{s',r} p(s', r \mid s, r) \left[r + \gamma E_\pi \left[G_{t+1} \mid S_{t+1} = s \right] \right].$$

$$\Rightarrow \boxed{V_\pi(s) = \sum_a \pi(a/s) \sum_{s',r} p(s', r \mid s, r) \left[r + \gamma V_\pi(s') \right].}$$

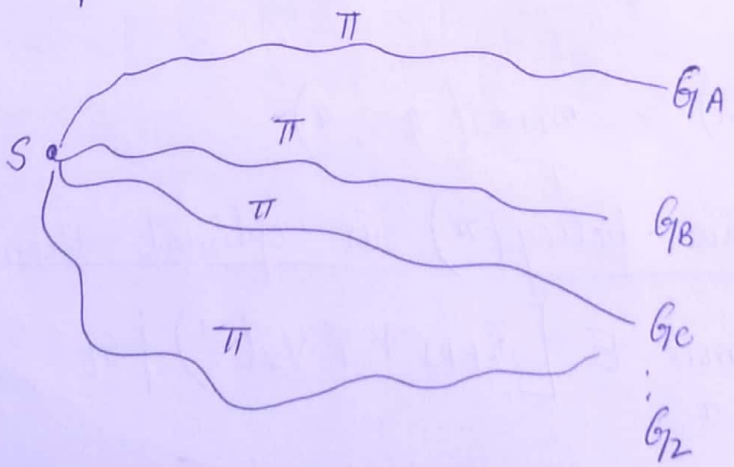
- $V_\pi(s)$: Value of current state
- $\pi(a/s)$: Policy
- $p(s', r \mid s, r)$: dynamics.
- $V_\pi(s')$: Value of successive state

It is Bellman Equation (Recursive Equation).

Simply. $V(s) = \max_a \{ (R(s,a) + \gamma V(s')) \}$

(Recursive relation between current state's value & successive state's value).

for example



Total (26 trajectory)

$$\therefore \boxed{V_\pi(s) = \frac{1}{26} \Sigma \left(G_A + G_B + G_C \cdots + G_Z \right)}$$

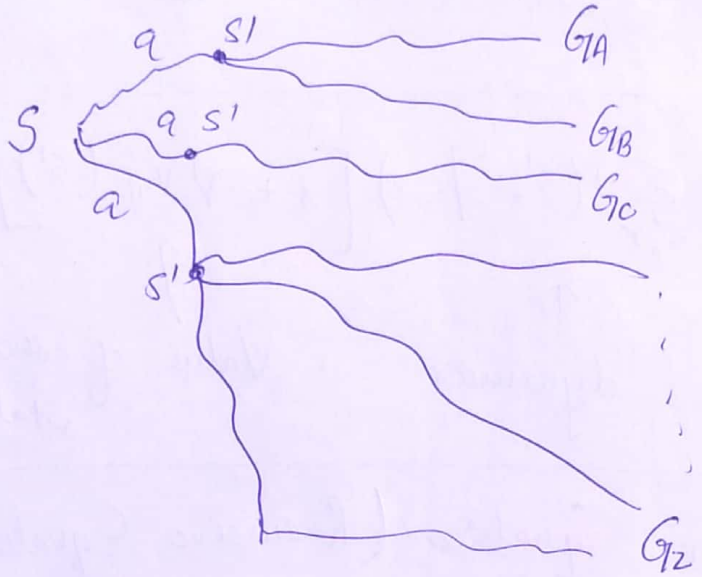
Average.

Scanned with CamScanner

Value of an Action

(with respect to a policy)

$$q_\pi(s,a) = E_\pi\left[G_t \mid s_t = s_1 , A_t = a \right].$$



$$\hat{q}_\pi(s,a) = \frac{1}{26} \sum \left(G_{1A} + G_{1B} + \dots G_{12} \right).$$

Optimal State Value & Optimal action value

Optimal → Maximum possible value of state/Action.

$$V_*(s) = \max_\pi V_\pi(s).$$

$$q_*(s,a) = \max_\pi q(s,a).$$

So if our policy (π) is optimal then.

$$V_*(s) = \max_a E\left[R_{t+1} + \gamma V_*(s') \mid s_t = s,\ A_t = a \right]$$

$$V_*(s) = \max_a \sum_{s,r} P(s', r \mid s, a)\left[r + \gamma V_*(s') \right].$$

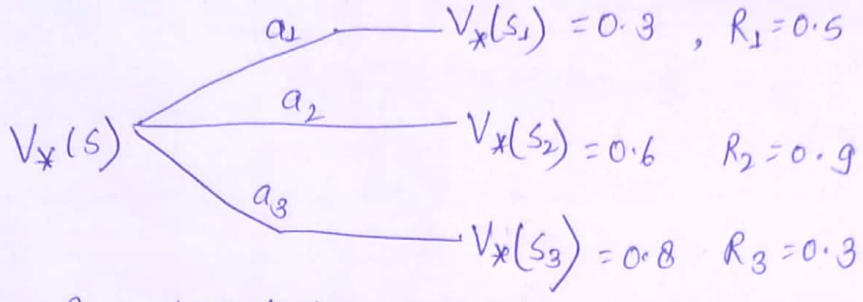
↑ Bellman optimality equation for state-action.

Scanned with CamScanner

Topic: Bellman Equation.

Let's take an example for better understanding:

$\gamma = 1.0$



$r = R_1 + \gamma V_*(s_1) = 0.8$

$r = R_2 + \gamma V_*(s_2) = 1.5 \leftarrow$ max.

$r = R_3 + \gamma V_*(s_3) = 1.1$

So $\boxed{V_*(s) = R_2 + \gamma V_*(s_2) = 1.5.}$

Scanned with CamScanner

☆ Policy Iteration & Value Iteration :-

Two reinforcement learning problems.

1) PREDICTION. (estimate accurate values)

2) CONTROL (find optimal Policy)

→ Policy iteration & Value iteration algorithms are dynamic programming algorithm that provides solution for it.

☆ Policy Iteration :-

It has 2 parts

1. Policy Evaluation ⇒ Compute values for the states.

2) Policy improvement ⇒ Improve the policy.

$\pi_0 \xrightarrow{\text{evaluate}} V\pi_0 \xrightarrow{\text{improve}} \pi_1 \xrightarrow{\text{evaluate}} V\pi_1 \xrightarrow{\text{improve}} \ldots \pi_*$

☆ Iterative Policy Evaluation :-

π ⟶ The policy to be evaluated

$\pi(s) \{ a_1 \text{ if } s = s_1, a_2 \text{ if } s = s_2, a_3 \text{ if } s = s_3 \ldots \}$

Step-1. Set arbitary value for the set.

Name of Lecturer :

2. Using π compute new values for the state

$$V_{new}(s) \leftarrow \sum_{s',r} P(s', r \mid s, \pi(s)) \left[r + V_{old}(s') \right]$$

for all states.

3. Repeat from (2) until convergence of the state values.

4. Policy Improvement :-

for all states $s \in S$ update $\pi(s)$ as

$$\pi(s) \leftarrow \underset{a}{\text{argmax}} \sum_{s',r} P(s', r \mid s, a) \left[r + \gamma V(s') \right]$$

r : next Reward; $\gamma V(s')$: discounted value of next state

Policy Iteration algo :

Repeat

  Repeat

    for all $s \in S$.

$$V(s) \leftarrow \sum_{s',r} P(s', r \mid s, \pi(s)) \left[r + \gamma V(s') \right]$$

($V(s')$: $V_{old}(s')$)

  until convergence.

  for all $s \in S$.

$$\pi(s) \leftarrow \underset{a}{\text{argmax}} \sum_{s',r} P(s', r \mid s, a) \left[r + \gamma V(s') \right]$$

until convergence

Return $\pi(s)$.

Scanned with CamScanner

★ Value Iteration :-

If we combine these two step into one step then it is called value Iteration

Algo

- Repeat

  for all $s \in S$.

  $$V(s) \leftarrow \max_a \sum_{s',r} P(s',r|s,a)\left[r + \gamma V_{old}(s')\right]$$

until Convergence.

$V_*(s) = V(s)$ for all $s \in S$

At the end of this loop we get optimal value $V_*(s)$ for all $s \in S$.

Return $$\pi(s) = \arg\max_a \sum_{s',r} P(s',r|s,a)\left[r + \gamma V_*(s')\right]$$

Optimal Policy.

Limitations :-

Algorithm steps in easier terms :-

1) Initialize a table V of value estimates for each square with all zeroes

2) Loop over every possible state s.

3) From state s loop over every possible action a

4) Get a list of all (probability, reward, s') transition tuples from state s, action a

5) Expected_reward = sum of all possible rewards multiplied by their probabilities

6) Expected_value = lookup V[s'] for each possible s', multiply by probability, sum.

7) Action_value = expected_reward + GAMMA * expected_value

8) Set V[s] to best action_value found.

9) Repeat steps 2-8 until the largest change in V[s] between iterations is below our threshold.

~~Arinda~~

Scanned with CamScanner

Policy Iteration & Value Iteration using monte-carlo.

→ In dynamic programming approach we assume that agent know everything about the environment that is It knows the transition dynamics it means agent knows the consequences of the action it takes on the environment.

→ But in real world application it is not possible to already know about the environment. (for ex. Robot).

→ So monte-carlo approach is where agent doesn't need to know about the environment, & it knows about the environment after interacting with the environ-ment & accordingly it can learn the policy to solve a particular task.

→ Monte-carlo method on the other hand is a very simple concept where agent learn about the states & reward when it interacts with the environment

→ In this method agent generate experienced samples & then based on average return, value is calculated for a state or state-action.

Key points of Monte-Carlo method:-

There is no model (agent does not know state MDP transitions).

- Agent learns from sampled experience.
- Learn state value $V_\pi(s)$ under policy π by experiencing average return from sampled ~~experience~~ episodes (value = average return).
- Only after a complete episode, values are updated (because of this algo convergence is slow and update happens after an episode is complete).
- There is no bootstrapping.
- Only can be used in episodic problems.

There are two types of monte-carlo methods:

- <u>First visit monte-carlo method</u> :-

In this case in an episode first visit of the state is counted (even if agent comes back to the same state multiple time in the episode), only first visit will be counted).

→ To evaluate state s, first we set number of visit, $N(s) = 0$, Total return $TR(s) = 0$. (updated across updates)

→ The first time-step t that state s is visited in an episode, increament counter $N(s) = N(s) + 1$.

Scanned with CamScanner

→ Increament total return $TR(s) = TR(s) + G_t$.

→ Value is estimated by means return $V(s) = TR(s)/N(s)$

→ By law of large numbers, $V(s) \rightarrow V_\pi(s)$
{this is called true value under policy π}.

Every visit Monte-Carlo

In an episode every visit of the state is counted

→ To evaluate state s, first we set number of visit, $N(s) = 0$, Total return $TR(s) = 0$.

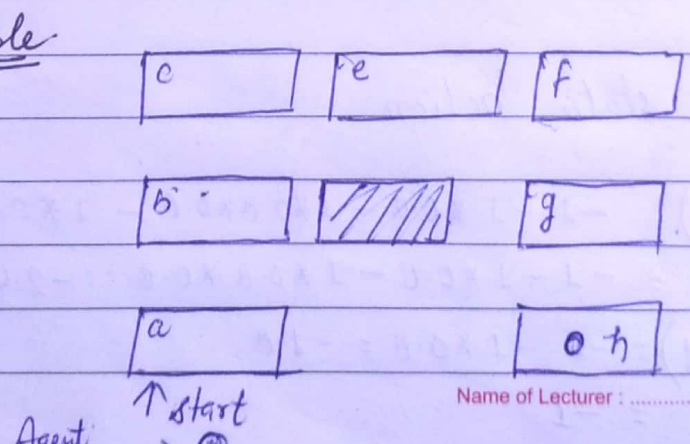
→ Every time step t that state s is visited in an episode, increament counter $N(s) = N(s) + 1$.

→ Increament total return $TR(s) = TR(s) + G_t$.

→ Value is estimated by mean return $V(s) = TR(s)/N(s)$

→ By law of large numbers, $V(s) \rightarrow V_\pi(s)$.

for Example



Name of Lecturer : .....

→ Agent has to start from starting point & has to reach the goal (h) in minimum no. of steps. (optimal policy).

State - Action Pair :-

{(a, up)
(b, up) (b, down)
(c, right) (c, down)
(d, down)
(e, up) (e, left) (e, right)
(f, left) (f, down)
(g, up) (g, down)}

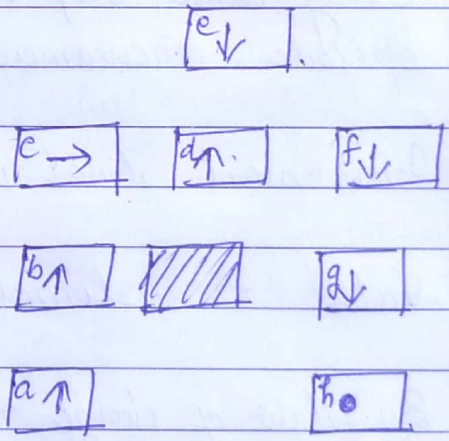
Goal State : h

Reward :

-1 : a, b, c, d, e, f, g.
100 : h.

Initial Policy. (Random).

a : up
b : up
c : right
d : down
e : up
f : down
g : down
h : stop.



Random state action

(e, down) $-1 - 1*0.8 - 1*0.8*0.8 - 1*0.8*0.8*0.8*0.8$.
(b, up) $= -1 - 1*0.8 - 1*0.8*0.8 = -2.44$
(c, right) $= -1 - 1*0.8 = -1.8$
(e, up) $= -1$

Scanned with CamScanner

again a Random episode

(e, right) = [98.2]
(f, down) = [99]
(g, down) = [100].

again Random action episode (b, c, e, d, e)

(b, up) = [-2.44, -2.95]

(c, right) = [-1.8, -2.44].
(d, down) = [-1]

finally calculate Q value.

Q(e, right) = 98.2.
Q(f, down) = 99
Q(g, down) = 100
Q(d, down) = -1
Q(c, down) = -2.95
Q(b, up) = -2.695
Q(c, right) = -2.12
Q(e, up) = -1.4.

Q value(e, right) > Q value(e, up).

optimal Policy.

We will update policy as.

a : up
d : down
g : down

b : up
e : right.
h : stop.

c : right
f : down

Ans.

Name of Lecturer : .....

Advantages :-

→ Zero bias
→ Good convergence properties.
→ Not very sensitive to initial value
→ Very simple to understand & use.

Limitations :-

→ It must wait until end of episode before return is known
→ It has high variance
→ It can only learn from complete sequences.
→ It only works for episodic (terminating) environments.

Scanned with CamScanner

★ Q-Learning :-

→ The idea behind Q-Learning is to have q-values for every action you/agent could possibly take given a state & over time agent will update those Q-values in such a way that running through a chain of actions will produce good results.

→ Q-learning algorithm is not environment specific. it is applicable to any environment.

→ It is a model free reinforcement learning algo.

→ It is a value-based learning algorithm. Value based algos update algos updates the value function based on an equation (Bellman equation), whereas other type policy-based estimates the value function with a greedy policy obtained from the last policy improvement

→ Q-learning is an off-policy learner. Means it learns the value of the optimal policy independently of the agent's actions.

On the other hand, an on-policy learner learns the value of the policy being carried out by the agent, including the exploration steps and it will find a policy that is optimal, taking into account the exploration inherent in the policy.

Scanned with CamScanner

What is this Q?

→ The 'Q' in Q-learning stands for quality.

→ Quality here represents how useful a given action is in gaining some future reward.

Q-Learning Definition

- $Q^*(s, a)$ is the expected value (cumulative discounted reward) of doing action a in state s and then following the optimal policy.
- Q-learning uses Temporal Differences (TD) to estimate the value of $Q^*(s, a)$. Temporal Difference is an agent learning from an environment through episodes with no prior knowledge of the environment.
- The agent maintains a table of $Q(S, A)$, where S is the set of states and A is the set of actions.
- $Q[s, a]$ represents its current estimates of $Q^*(s, a)$

Q-Table :- Q-Table is the data structure used to calculate the maximum expected future rewards for action at each state

Basically this table will guide us to the best action at each state. To know Q-value, ~~each~~ of the Q-table, Q-learning algorithm is used.

Name of Lecturer : .....

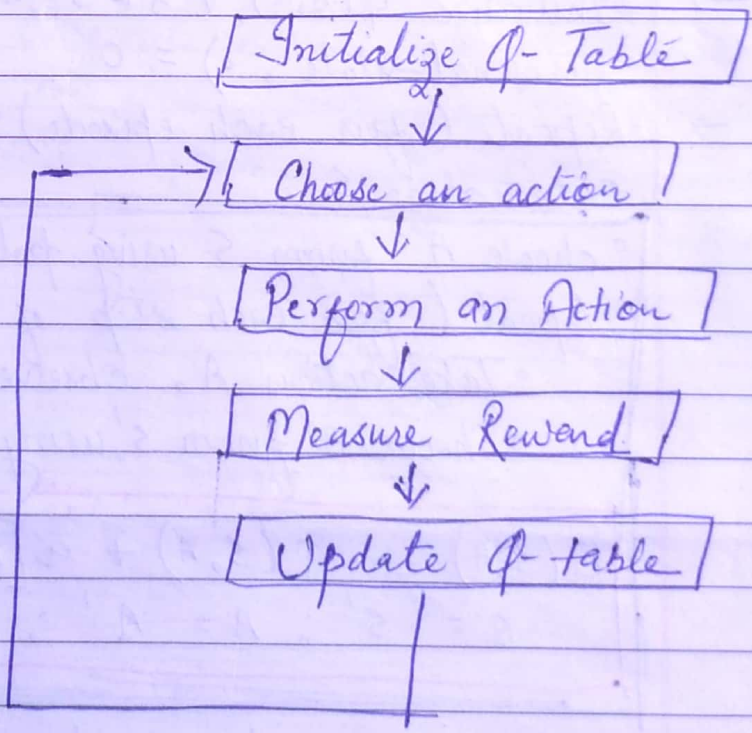
Q- function :-

The Q- function uses Bellman equation & takes two inputs : state (s) and action (a)

$$Q^{\pi}(s_t, a_t) = E[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \cdots \cdots | s_t, a_t]$$

- $Q^{\pi}(s_t, a_t)$: Q-values for the state given a particular state
- $E[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \cdots \cdots]$: Expected discounted cumulative reward
- s_t, a_t : Given the state & action

Q- Learning Algorithm Process :- (Flow Chart)



A number of iterations result a good Q-table.

Expected future reward

Discount Rate

$$\text{New } Q(s,a) = Q(s,a) + \alpha\left[R(s,a) + \gamma \max Q'(s',a') - Q(s,a)\right]$$

- New Q value for the state & action
- Current Q-values
- Learning Rate
- Reward for taking an action in a state
- Maximum expected future reward
- Current Q-values

Algo Steps :-

→ Initialize $Q(s,a)$, $\forall s \in S$, $a \in A(s)$, arbitrarily, and $Q($ terminal-state, $\cdot) = 0$

→ Repeat (for each episode):

- Initialize S
- choose A from S using policy derived from Q (e.g., ϵ-greedy)
- Repeat (for each step of episode):
  - Take action A, observe R, S'
  - Choose A' from S' using policy derived from Q (eg- ϵ-greedy)

$$Q(S,A) = Q(S,A) + \alpha\left[R + \gamma Q(S',A') - Q(S,A)\right]$$

$S = S'$; $A = A'$;

until S is terminal.

Algorithm for State-Action-Reward-State Action.

Scanned with CamScanner

★ State-Action-Reward-State-Action (SARSA)

→ The Sarsa algorithm is an On-policy algorithm for Temporal difference learning

→ The major difference between it and Q-learning, is that maximum reward for the next state is not neccesarily used for updating the Q-values.

Instead a new action, and therefore reward is selected using the same policy that determined the original action.

→ The name Sarsa actually comes from the fact that the updates are done using the quintuple $Q(s, a, r, s', a')$ where: s, a are the original state and action, r is the reward observed in the following state and s', a' are the new state-action pair.

∈-Greedy Policy :-

1) Generate a random number $r \in [0, 1]$

2) If $r < \epsilon$ choose an action derived from the Q values (which yields the maximum utility)

3) Else choose a random action.

The value of ϵ determines the exploration-exploitation of the agent.

If ε is large, the random number r will hardly ever be larger than ε and a random action will hardly ever be taken (less exploration, more exploitation)

If ε is small, the random number r will often be larger than ε which will cause the agent to choose more random action. This characteristic will allow the agent to explore the environment even more.

Algo steps

Initialize $Q(s,a)$, for all $s \in S$, $a \in A(s)$, arbitarily, and $Q(\text{terminal-state}, \cdot) = 0$.
Repeat (for each episode):
  Initialize S.
  Repeat (for each step of episode):
  choose A from S using policy derived from Q (eg ε-greedy)
  Take action A, observe R, S'

$$Q(S,A) = Q(S,A) + \alpha [R + \gamma \max_a Q(S',a) - Q(S,A)].$$

  $S \leftarrow S'$
until S is terminal.

Q learning off policy TD control Algo

Scanned with CamScanner

Model-Based Reinforcement Learning :-

→ Reinforcement learning maximizes rewards for our actions

→ From the equations mentioned below, rewards depends on the policy & the system dynamics (model)

$$p_\theta(s_1, a_1, \ldots s_T, a_T) = p(s_1) \prod_{t=1}^{T} \pi_\theta(a_t|s_t)\, p(s_{t+1}|s_t, a_t)$$

↑ Policy ↑ Model.

$$\max_\theta E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(s_t, a_t) \right] \leftarrow \text{Rewards.}$$

→ In model-free RL, we ignore the model. We depend on sampling and simulation to estimate rewards so we don't need to know the inner working of the system

→ In Model-based RL, if we can define a cost function ourselves, we can calculate the optimal actions using the model directly.

→ Model free RL ignores the model & care less about the inner working.

Name of Lecturer : .....

$P(s_{t+1} | s_t, a_t)$, $\pi_\theta(a_t | s_t)$

$\pi_\theta(u_t | x_t) \rightarrow \tau = \{x_1, u_1, x_2, u_2 \ldots x_T, u_T\}$

$$\max_{\pi_\theta} E_{\tau \sim \pi_\theta} \left[\sum_t \gamma^t r(x_t, u_t) \right]$$

a_t s_t

↑ Model free RL

→ On the contrary model based RL focuses on the model.

$P(s_{t+1} | s_t, a_t)$, $\pi_\theta(a_t | s_t)$

↑ Model.

$\tau = \{x_1, u_1, x_2, u_2 \ldots x_T, u_T\}$

$$\min_{u_1 \ldots u_T} \sum_{t=1}^{T} \underbrace{c(x_t, u_t)}_{\text{Cost}} \quad s.t. \quad x_t = \underbrace{f(x_{t-1}, u_{t-1})}_{\text{Model.}}$$

↑ Model Based RL

★ Known Models :- Many Games like alphaGo

→ Vehicle dynamics model.

→ Kinematic Bicycle model.

Scanned with CamScanner

Learn the model

→ In model based RL, the model may be known or learned.

→ When it is learned we run a base policy, like a or any educated policy, and observe the trajectory.

→ Then we fit a model using this sampled data.

1) Run base policy $\pi_0(a_t|s_t)$ {eg. random policy} to collect $\mathcal{D} = \{(s, a, s')_i\}$.

2) Learn dynamics model $f(s,a)$ to minimize

$$\sum_i \| f(s_i, a_i) - s'_i \|^2$$

3) Plan through $f(s,a)$ to choose actions.

4) Execute those actions and add the resulting data $\{(x, u, x')_j\}$ to $\mathcal{D}$.

5) append (s, a, s') to dataset $\mathcal{D}$

↑ Every N steps

→ Model based RL has a strong advantage of being sample efficient.

→ It requires very few samples to learn

→ On policy Gradient methods can take 10M training iterations while Model based RL is in range of hundereds.

→ for ex, To train a physical robot for a simple task, a Model based method may take ~~weeks~~ 20 minutes while a policy gradient method may takes weeks.

→ Model based methods have more assumptions & approximation & therefore, limit the trained data model to fewer tasks

Better sample Efficient ← → Less sample Efficient

Model Based.	Off Policy Q-learning	(Actor-critic)	On policy Policy Gradient	Evolutionary Gradient-Free
(100 time steps).	(1M Time Steps)		(10M time steps).	(100M time steps).

Scanned with CamScanner